arXiv:2307.16789v2 [cs.AI] 3 Oct 2023

# TOOLLLM: FACILITATING LARGE LANGUAGE MODELS TO MASTER 16000+ REAL-WORLD APIS

**Yujia Qin**[1*], **Shihao Liang**[1*], **Yining Ye**[1], **Kunlun Zhu**[1], **Lan Yan**[1], **Yaxi Lu**[1], **Yankai Lin**[3†], **Xin Cong**[1], **Xiangru Tang**[4], **Bill Qian**[4], **Sihan Zhao**[1], **Lauren Hong**[1], **Runchu Tian**[1], **Ruobing Xie**[5], **Jie Zhou**[5], **Mark Gerstein**[4], **Dahai Li**[2,6], **Zhiyuan Liu**[1†], **Maosong Sun**[1†]
[1]Tsinghua University [2]ModelBest Inc. [3]Renmin University of China
[4]Yale University [5]WeChat AI, Tencent Inc. [6]Zhihu Inc.
yujiaqin16@gmail.com

## ABSTRACT

Despite the advancements of open-source large language models (LLMs), e.g., LLaMA, they remain significantly limited in tool-use capabilities, i.e., using external tools (APIs) to fulfill human instructions. The reason is that current instruction tuning largely focuses on basic language tasks but ignores the tool-use domain. This is in contrast to the excellent tool-use capabilities of state-of-the-art (SOTA) closed-source LLMs, e.g., ChatGPT. To bridge this gap, we introduce ToolLLM, a general tool-use framework encompassing data construction, model training, and evaluation. We first present ToolBench, an instruction-tuning dataset for tool use, which is constructed automatically using ChatGPT. Specifically, the construction can be divided into three stages: (i) API collection: we collect $16,464$ real-world RESTful APIs spanning $49$ categories from RapidAPI Hub; (ii) instruction generation: we prompt ChatGPT to generate diverse instructions involving these APIs, covering both single-tool and multi-tool scenarios; (iii) solution path annotation: we use ChatGPT to search for a valid solution path (chain of API calls) for each instruction. To enhance the reasoning capabilities of LLMs, we develop a novel depth-first search-based decision tree algorithm. It enables LLMs to evaluate multiple reasoning traces and expand the search space. Moreover, to evaluate the tool-use capabilities of LLMs, we develop an automatic evaluator: ToolEval. Based on ToolBench, we fine-tune LLaMA to obtain an LLM ToolLLaMA, and equip it with a neural API retriever to recommend appropriate APIs for each instruction. Experiments show that ToolLLaMA demonstrates a remarkable ability to execute complex instructions and generalize to unseen APIs, and exhibits comparable performance to ChatGPT. Our ToolLLaMA also demonstrates strong zero-shot generalization ability in an out-of-distribution tool-use dataset: APIBench. The codes, trained models, and demo are publicly available at https://github.com/OpenBMB/ToolBench.

## 1 INTRODUCTION

Tool learning (Qin et al., 2023b) aims to unleash the power of large language models (LLMs) to effectively interact with various tools (APIs) to accomplish complex tasks. By integrating LLMs with APIs, we can greatly expand their utility and empower them to serve as efficient intermediaries between users and the vast ecosystem of applications. Although open-source LLMs, e.g., LLaMA (Touvron et al., 2023a), have achieved versatile capabilities through instruction tuning (Taori et al., 2023; Chiang et al., 2023), they still lack the sophistication in performing higher-level tasks, such as appropriately interacting with tools (APIs) to fulfill complex human instruction. This deficiency is because current instruction tuning largely focuses on basic language tasks, with a relative neglect of the tool-use domain. On the other hand, current state-of-the-art (SOTA) LLMs (e.g., ChatGPT (OpenAI,

[*] Indicates equal contribution.
[†] Corresponding author.

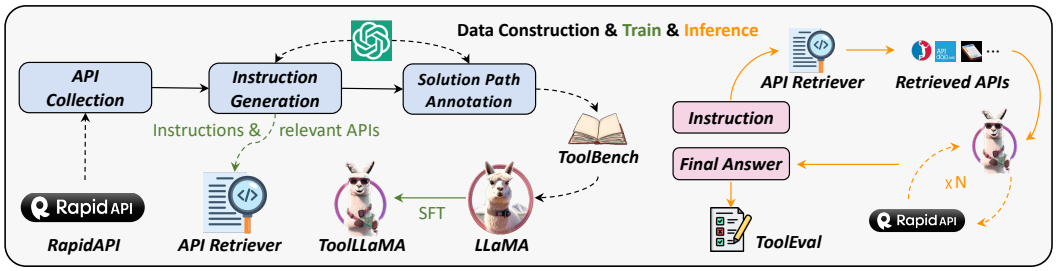


Figure 1: Three phases of constructing ToolBench and how we train our API retriever and ToolLLaMA. During inference of an instruction, the API retriever recommends relevant APIs to ToolLLaMA, which performs multiple rounds of API calls to derive the final answer. The whole reasoning process is evaluated by ToolEval.

2022) and GPT-4 (OpenAI, 2023)), which have demonstrated impressive competencies in utilizing tools (Bubeck et al., 2023), are closed-source with their inner mechanisms opaque. This limits the democratization of AI technologies and the scope of community-driven innovation and development. In this regard, we deem it urgent to *empower open-source LLMs to skillfully master diverse APIs.*

Although prior works have explored building instruction tuning data for tool use (Li et al., 2023a; Patil et al., 2023; Tang et al., 2023; Xu et al., 2023b), they fail to fully stimulate the tool-use capabilities within LLMs and have inherent limitations: (1) **limited APIs**: they either fail to involve real-world APIs (e.g., RESTAPI) (Patil et al., 2023; Tang et al., 2023) or consider only a small scope of APIs with poor diversity (Patil et al., 2023; Xu et al., 2023b; Li et al., 2023a); (2) **constrained scenario**: existing works are confined to instructions that only involve one single tool. In contrast, real-world scenarios may require that multiple tools are interleaved together for multi-round tool execution to solve a complex task. Besides, they often assume that users manually specify the ideal API set for a given instruction in advance, which is infeasible with a large collection of real-world APIs; (3) **inferior planning and reasoning**: existing works adopted either CoT (Wei et al., 2023) or ReACT (Yao et al., 2022) for model reasoning, which cannot fully elicit the capabilities stored in LLMs and thus fail to handle complex instructions. In addition, some works do not even execute APIs to obtain real responses (Patil et al., 2023; Tang et al., 2023), which serve as important information for subsequent model planning.

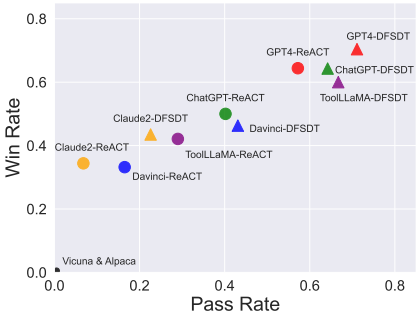


Figure 2: Pass rate (↑) and win rate (↑) of different methods in tool-use evaluation. For win rate, we compare each method with ChatGPT-ReACT. DFSDT is our improved reasoning strategy over ReACT. ToolLLaMA surpasses Text-Davinci-003, Claude-2, and almost performs on par with ChatGPT.

To facilitate tool-use capabilities within open-source LLMs, we introduce **ToolLLM**, a general tool-use framework including data construction, model training, and evaluation. As illustrated in Figure 1, we collect a high-quality instruction-tuning dataset **ToolBench**. It is constructed automatically using ChatGPT (*gpt-3.5-turbo-16k*), which has been upgraded with function call (link) capabilities. The comparison between ToolBench and prior works is listed in Table 1. Specifically, the construction of ToolBench entails three phases:

- **API Collection**: we gather **16,464** representational state transfer (REST) APIs from RapidAPI (link), a platform that hosts massive real-world APIs provided by developers. These APIs span **49** diverse categories such as social media, e-commerce, and weather. For each API, we crawl detailed API documents from RapidAPI, including the functionality descriptions, required parameters, code snippets for API calls, etc. By comprehending these documents to learn to execute APIs, LLMs can generalize to new APIs unseen during training;
- **Instruction Generation**: we first sample APIs from the whole set and then prompt ChatGPT to generate diverse instructions for these APIs. To cover practical scenarios, we curate instructions

| Resource | **ToolBench** (this work) | **APIBench** (Patil et al., 2023) | **API-Bank** (Li et al., 2023a) | **ToolAlpaca** (Tang et al., 2023) | **ToolBench** (Xu et al., 2023b) |
|---|---|---|---|---|---|
| Real-world API? | ✓ | ✗ | ✓ | ✗ | ✓ |
| Real API Call&Response? | ✓ | ✗ | ✓ | ✗ | ✓ |
| Multi-tool Scenario? | ✓ | ✗ | ✗ | ✗ | ✗ |
| API Retrieval? | ✓ | ✓ | ✗ | ✗ | ✓ |
| Multi-step Reasoning? | ✓ | ✗ | ✓ | ✓ | ✓ |
| Number of tools | **3451** | 3 | 53 | 400 | 8 |
| Number of APIs | **16464** | 1645 | 53 | 400 | 232 |
| Number of Instances | **126486** | 17002 | 274 | 3938 | 2746 |
| Number of Real API Calls | **469585** | 0 | 568 | 0 | 3926 |
| Avg. Reasoning Traces | 4.0 | 1.0 | 2.1 | 1.0 | **5.9** |

Table 1: A comparison of our ToolBench to notable instruction tuning dataset for tool learning.

that involve both **single-tool** and **multi-tool** scenarios. This ensures that our model learns not only how to interact with individual tools but also how to combine them to accomplish complex tasks;

- **Solution Path Annotation**: each solution path may contain multiple rounds of model reasoning and real-time API calls to derive the final response. However, even the most sophisticated LLM, i.e., GPT-4, achieves a low pass rate for complex human instructions, making annotation inefficient. To this end, we develop a novel **depth-first search-based decision tree** (DFSDT) to bolster the planning and reasoning ability of LLMs. Compared with conventional ReACT, DFSDT enables LLMs to evaluate a multitude of reasoning paths and make deliberate decisions to either retract steps or proceed along a promising path. In experiments, DFSDT significantly improves the annotation efficiency and successfully completes those complex instructions that cannot be fulfilled using ReACT.

To assess the tool-use capabilities of LLMs, we develop an automatic evaluator, **ToolEval**, backed up by ChatGPT. It comprises two key metrics: (1) *pass rate*, which measures LLM's ability to successfully execute an instruction within limited budgets, and (2) *win rate*, which compares the quality and usefulness of two solution paths. We demonstrate that ToolEval achieves a high correlation with human evaluation and provides a robust, scalable, and reliable assessment for machine tool use.

By fine-tuning LLaMA on ToolBench, we obtain **ToolLLaMA**. After evaluation based on our ToolEval, we derive the following findings:

- ToolLLaMA demonstrates a compelling capability to handle both single-tool and complex multi-tool instructions. As depicted in Figure 2, ToolLLaMA outperforms Text-Davinci-003 and Claude-2, achieves comparable performance to the "teacher model" ChatGPT, and is only slightly inferior to GPT4. Besides, ToolLLaMA exhibits **robust generalization to previously unseen APIs**, requiring only the API documentation to adapt to new APIs effectively. This flexibility allows users to incorporate novel APIs seamlessly, thus enhancing the model's practical utility.
- We show that our DFSDT serves as a general decision-making strategy to enhance the reasoning capabilities of LLMs. DFSDT broadens the search space by considering multiple reasoning traces and achieves significantly better performance than ReACT.
- We train a neural **API retriever**, which alleviates the need for manual selection from the large API pool in practice. As shown in Figure 1, given an instruction, the API retriever recommends a set of relevant APIs, which are sent to ToolLLaMA for multi-round decision making to derive the final answer. Despite sifting through a large pool of APIs, the retriever exhibits remarkable retrieval precision, returning APIs closely aligned with the ground truth.
- ToolLLaMA exhibits strong **generalization** performance on an **out-of-distribution** (OOD) dataset APIBench (Patil et al., 2023). Despite not training on any of the APIs or instructions on APIBench, ToolLLaMA performs on par with Gorilla, a pipeline specifically designed for APIBench.

## 2 Dataset Construction

We introduce the three-stage construction process of ToolBench: API collection (§ 2.1), instruction generation (§ 2.2), and solution path annotation (§ 2.3). All procedures are based on ChatGPT (*gpt-3.5-turbo-16k*), requiring minimal human supervision and can be easily extended to new APIs.